Delta Exchange API Documentation — As Used in This Algo

> **NOTE:** This document is extracted entirely from your codebase. It covers every Delta Exchange REST API endpoint, authentication mechanism, data model, and configuration used by your trading algorithm.

Table of Contents

1. Base URLs & Environments
2. Authentication (HMAC-SHA256)
3. Rate Limiting
4. Symbol & Contract Conventions
5. Public Endpoints (No Auth Required)
6. Private Endpoints (Auth Required)
7. Heartbeat / Deadman Switch
8. Response Format
9. Data Models
10. Error Handling & Error Codes
11. Configuration Reference

---

1. Base URLs & Environments

Environment	Base URL
Production (India)	`https://api.india.delta.exchange`
Testnet	`https://cdn-ind.testnet.deltaex.org`

> **TIP:** Get testnet API keys at: https://testnet.delta.exchange → Account → API Keys

Configured in [config.yaml](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/config/config.yaml):

```
exchange:
  environment: "production"          # "production" | "testnet"
  base_url_production: "https://api.india.delta.exchange"
  base_url_testnet:    "https://cdn-ind.testnet.deltaex.org"
  api_key:    "YOUR_API_KEY_HERE"
  api_secret: "YOUR_API_SECRET_HERE"
  user_agent: "delta-options-algo-v1"
```

Environment variables override config for secrets:

- `DELTA_API_KEY`
- `DELTA_API_SECRET`
- `DELTA_BASE_URL`

---

2. Authentication (HMAC-SHA256)

All **private endpoints** require HMAC-SHA256 signed headers. Public endpoints do not.

Signature Generation

Source: [client.py#L67-L78](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L67-L78)

```
timestamp = str(int(time.time()))
message   = METHOD + timestamp + path + query_string + body
signature = hmac.new(
    bytes(api_secret, "utf-8"),
    bytes(message, "utf-8"),
    hashlib.sha256
).hexdigest()
```

Required Headers

Header	Value
`api-key`	Your API key
`timestamp`	Unix timestamp (seconds)
`signature`	HMAC-SHA256 hex digest
`User-Agent`	e.g. `delta-options-algo-v1`
`Content-Type`	`application/json`
`Accept`	`application/json`

Signature Message Format

```
{METHOD}{TIMESTAMP}{PATH}{QUERY_STRING}{BODY}
```

- **METHOD**: Uppercase HTTP method (`GET`, `POST`, `PUT`, `DELETE`)
- **TIMESTAMP**: Unix epoch seconds as string
- **PATH**: e.g. `/v2/orders`
- **QUERY_STRING**: URL query string including `?` (e.g. `?product_id=123`)
- **BODY**: JSON body with compact separators (`,`, `:`) or empty string

3. Rate Limiting

Source: [client.py#L21-L39](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L21-L39)

Quota

- **9,000 requests per 5 minutes** (rolling window)
- When a `429` response is received, the algo reads `X-RATE-LIMIT-RESET` header (milliseconds) and sleeps

Endpoint Weights

Endpoint Category	Weight
`place_order`	5
`edit_order`	5
`cancel_order`	5
`get_orders`	3
`get_positions`	3
`get_balance`	3
`get_tickers`	3
`get_products`	3
`get_orderbook`	3
`batch_orders`	25
`ohlc_candles`	3
`default`	1

Retry Strategy

- Max retries: **3** (configurable)
- Retry delay: **2s × attempt** (exponential backoff)
- On `5xx` errors: auto-retry
- On `429`: sleep for `X-RATE-LIMIT-RESET` ms (min 5s)

4. Symbol & Contract Conventions

Option Symbol Format

```
{TYPE}-{UNDERLYING}-{STRIKE}-{EXPIRY}
```

Component	Example	Notes
TYPE	`C` or `P`	Call or Put
UNDERLYING	`BTC`, `ETH`	Underlying asset
STRIKE	`90000`	Strike price
EXPIRY	`310125`	`DDMMYY` format

Full example: `C-BTC-90000-310125` = BTC Call, strike 90000, expiring 31-Jan-2025

Spot Symbol

```
{UNDERLYING}USD
```

Example: `BTCUSD`, `ETHUSD`

Contract Types

API Value	Description
`call_options`	Call options
`put_options`	Put options

Expiry Date Format for API Params

- **In symbols**: `DDMMYY` (e.g. `310125`)
- **In API query params** (e.g. `expiry_date`): `DD-MM-YYYY` (e.g. `31-01-2025`)

---

5. Public Endpoints (No Auth Required)

5.1 GET `/v2/assets`

Purpose: List all available assets. Used as a connectivity check (`ping()`).

Source: [client.py#L128](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L128)

```
client.get_assets()
```

5.2 GET `/v2/products`

Purpose: List all tradeable products (options, futures, etc.)

Source: [client.py#L129-L133](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L129-L133)

Parameter	Type	Default	Description
`contract_types`	string	—	Comma-separated: `call_options,put_options`
`states`	string	`live`	Product states to filter
`expiry`	string	—	Expiry date filter
`page_size`	int	`100`	Results per page

Response fields (per product):

- `symbol` — e.g. `C-BTC-90000-310125`
- `product_id` — numeric ID used for order placement
- `contract_type` — `call_options` or `put_options`
- `tick_size` — minimum price increment (e.g. `0.5`)
- `strike_price` — strike price

```
# Get all live options
products = client.get_products(
    contract_types="call_options,put_options",
    states="live"
)
```

5.3 GET `/v2/products/{symbol}`

Purpose: Get details for a specific product by symbol.

Source: [client.py#L134](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L134)

```
product = client.get_product_by_symbol("C-BTC-90000-310125")
```

5.4 GET `/v2/tickers`

Purpose: Get market data (price, greeks, IV, quotes) for options. This is the **primary endpoint for option chain data**.

Source: [client.py#L135-L140](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L135-L140)

Parameter	Type	Description
`contract_types`	string	`call_options,put_options`

`underlying_asset_symbols`	string	e.g. `BTC`
`expiry_date`	string	`DD-MM-YYYY` format (e.g. `31-01-2025`)

Response fields (per ticker):

Field	Type	Description
`symbol`	string	e.g. `C-BTC-90000-310125`
`product_id`	int	Numeric product identifier
`contract_type`	string	`call_options` or `put_options`
`spot_price`	string	Current underlying spot price
`mark_price`	string	Mark price of the option
`mark_vol`	string	Mark implied volatility (percentage, e.g. `65.5`)
`oi`	number	Open interest
`oi_change_usd_6h`	number	OI change in USD (6-hour)
`strike_price`	string/number	Strike price
`greeks`	object	Greeks sub-object (see below)
`quotes`	object	Quotes sub-object (see below)

`greeks` object:

Field	Type	Description
`delta`	string/number	Option delta
`gamma`	string/number	Option gamma
`theta`	string/number	Option theta
`vega`	string/number	Option vega
`rho`	string/number	Option rho

`quotes` object:

Field	Type	Description
`best_bid`	string/number	Best bid price
`best_ask`	string/number	Best ask price
`bid_iv`	string/number	Bid implied volatility

`ask_iv`	string/number	Ask implied volatility

```
# Get BTC option chain for specific expiry
tickers = client.get_tickers(
    contract_types="call_options,put_options",
    underlying="BTC",
    expiry_date="31-01-2025"
)

# Shorthand for option chain
chain = client.get_option_chain("BTC", "31-01-2025")
```

5.5 GET `/v2/tickers/{symbol}`

Purpose: Get detailed ticker data for a single product.

Source: [client.py#L141](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L141)

Returns same fields as the bulk tickers endpoint but for one symbol.

```
ticker = client.get_ticker("C-BTC-90000-310125")
# Access: ticker["greeks"]["delta"], ticker["quotes"]["best_bid"], ticker["mark_price"]
```

5.6 GET `/v2/l2orderbook/{symbol}`

Purpose: Get Level 2 order book (bids and asks) for a product.

Source: [client.py#L144-L145](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L144-L145)

Parameter	Type	Default	Description
`depth`	int	`5`	Number of levels on each side

```
orderbook = client.get_orderbook("C-BTC-90000-310125", depth=5)
```

5.7 GET `/v2/history/candles`

Purpose: Get OHLC candlestick data for historical analysis (trend filter, ATR).

Source: [client.py#L146-L147](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L146-L147)

Parameter	Type	Description

`symbol`	string	e.g. `BTCUSD`
`resolution`	string	Candle resolution: `1h`, `1d`, etc.
`start`	int	Start Unix timestamp
`end`	int	End Unix timestamp

Response fields (per candle):

- `open`, `high`, `low`, `close` — OHLC prices

```
end = int(time.time())
start = end - 86400 * 30  # 30 days ago
candles = client.get_ohlc("BTCUSD", "1h", start, end)
```

6. Private Endpoints (Auth Required)

6.1 GET `/v2/profile`

Purpose: Get account profile (email, KYC status).

Source: [client.py#L166](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L166)

```
profile = client.get_profile()
# profile["email"], profile["is_kyc_done"]
```

6.2 GET `/v2/wallet/balances`

Purpose: Get wallet balances across all assets.

Source: [client.py#L150-L152](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L150-L152)

Response: List of balance objects:

Field	Type	Description
`asset_symbol`	string	e.g. `USDT`, `BTC`
`available_balance`	string	Available balance (not in margin)

```
balances = client.get_wallet_balances()
for b in balances:
    print(f"{b['asset_symbol']}: {b['available_balance']}")
```

6.3 GET `/v2/positions`

Purpose: Get positions, optionally filtered by product or underlying.

Source: [client.py#L153-L157](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L153-L157)

Parameter	Type	Description
`product_id`	int	Filter by specific product
`underlying_asset_symbol`	string	Filter by underlying (e.g. `BTC`)

Response fields (per position):

- `size` — Position size (negative = short)
- `product_id` — Product identifier

```
# By product
pos = client.get_positions(product_id=12345)

# By underlying
positions = client.get_positions(underlying="BTC")
```

6.4 GET `/v2/positions/margined`

Purpose: Get all margined positions across contract types.

Source: [client.py#L158-L159](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L158-L159)

Parameter	Type	Default	Description
`contract_types`	string	`call_options,put_options`	Filter by contract type

```
all_positions = client.get_all_positions()
# or
all_positions = client.get_all_positions(ct="call_options,put_options")
```

6.5 GET `/v2/orders`

Purpose: Get active/pending orders.

Source: [client.py#L160-L164](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L160-L164)

Parameter	Type	Default	Description
`states`	string	`open,pending`	Order states to filter
`product_ids`	string	—	Comma-separated product IDs
`contract_types`	string	`call_options,put_options`	Contract types
`page_size`	int	`50`	Max results

```
orders = client.get_active_orders()
```

6.6 GET `/v2/orders/{order_id}`

Purpose: Get details of a single order by ID.

Source: [client.py#L165](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L165)

Response fields:

- `id` — Order ID
- `state` — `open`, `closed`, `cancelled`, `pending`

```
order = client.get_order(order_id=123456)
```

6.7 POST `/v2/orders`

Purpose: Place a new order (limit, market, stop).

Source: [client.py#L168-L178](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L168-L178)

Request body:

Field	Type	Required	Description
`product_id`	int	✓	Numeric product ID
`size`	int	✓	Number of contracts
`side`	string	✓	`buy` or `sell`
`order_type`	string	✓	`limit_order` or `market_order`
`limit_price`	string	For limit	Limit price as string
`time_in_force`	string	No	`gtc` (default), `ioc`
`post_only`	bool	No	Post-only mode (default `false`)

`reduce_only`	bool	No	Close existing position only (default `false`)
`client_order_id`	string	No	Custom ID (max 32 chars)
`stop_order_type`	string	No	For stop orders
`stop_price`	string	No	Trigger price for stops

Response: Order object with `id`, `state`, etc.

```
# Limit order
order = client.place_order(
    product_id=12345,
    size=1,
    side="sell",
    order_type="limit_order",
    limit_price="150.50",
    time_in_force="gtc",
    client_order_id="my_order_001"
)

# Market order (shorthand)
order = client.place_market_order(
    product_id=12345,
    size=1,
    side="buy",
    reduce_only=True
)
```

6.8 PUT `/v2/orders`

Purpose: Edit/amend an existing order.

Source: [client.py#L183-L186](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L183-L186)

Request body:

Field	Type	Required	Description
`id`	int	✓	Order ID to edit
`product_id`	int	✓	Product ID
`size`	int	✓	New size
`limit_price`	string	No	New limit price

```
client.edit_order(order_id=123, product_id=456, size=2, limit_price="155.00")
```

6.9 DELETE `/v2/orders`

Purpose: Cancel a single order.

Source: [client.py#L188-L192](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L188-L192)

Request body:

Field	Type	Required	Description
`id`	int	✓	Order ID
`product_id`	int	✓	Product ID
`client_order_id`	string	No	Client order ID

```
client.cancel_order(order_id=123, product_id=456)
```

6.10 DELETE `/v2/orders/all`

Purpose: Cancel all open orders.

Source: [client.py#L194-L199](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L194-L199)

Request body:

Field	Type	Description
`cancel_limit_orders`	bool	Always `true`
`product_id`	int	(Optional) Cancel for specific product
`contract_types`	string	(Optional) Cancel for contract types

```
# Cancel all option orders
client.cancel_all_orders()

# Cancel for specific product
client.cancel_all_orders(product_id=12345)
```

6.11 POST `/v2/orders/batch`

Purpose: Place multiple orders in one request (must be same `product_id`).

Source: [client.py#L201-L202](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L201-L202)

Request body:

Field	Type	Description
`product_id`	int	Product ID (all orders must be same product)
`orders`	array	Array of order objects

> **WARNING:** Batch orders require all orders to have the **same `product_id`**. For multi-leg strategies (e.g., Iron Condor), each leg must be placed individually.

```
client.batch_place_orders(
    product_id=12345,
    orders=[
        {"size": 1, "side": "sell", "order_type": "limit_order", "limit_price": "150.00"},
        {"size": 1, "side": "buy",  "order_type": "limit_order", "limit_price": "200.00"},
    ]
)
```

6.12 POST `/v2/positions/close_all`

Purpose: Emergency close all open positions.

Source: [client.py#L211-L214](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L211-L214)

Request body:

Field	Type	Description
`close_all_portfolio`	bool	Close portfolio-margined positions
`close_all_isolated`	bool	Close isolated-margined positions
`user_id`	int	Set to `0`

```
client.close_all_positions()
```

6.13 POST `/v2/products/{product_id}/orders/leverage`

Purpose: Set leverage for a product.

Source: [client.py#L216-L217](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L216-L217)

```
client.set_leverage(product_id=12345, leverage=5)
```

7. Heartbeat / Deadman Switch

The heartbeat system is a safety mechanism: if the algo stops sending heartbeats, Delta Exchange automatically cancels all open orders.

Source: [heartbeat.py](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/live/heartbeat.py), [client.py#L219-L226](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L219-L226)

7.1 POST `/v2/heartbeat/create`

Purpose: Register a new heartbeat (deadman switch).

Request body:

Field	Type	Description
`heartbeat_id`	string	Unique identifier (e.g. `delta_algo_hb_001`)
`impact`	string	Always `contracts`
`contract_types`	array	`["call_options", "put_options"]`
`config`	array	Action config (see below)

Config entry:

Field	Type	Description
`action`	string	`cancel_orders` — what happens on miss
`unhealthy_count`	int	Number of missed beats before action

```
client.create_heartbeat(
    hb_id="delta_algo_hb_001",
    ct=["call_options", "put_options"],
    action="cancel_orders",
    unhealthy_count=1
)
```

7.2 POST `/v2/heartbeat`

Purpose: Send periodic heartbeat acknowledgment to keep the deadman switch alive.

Request body:

Field	Type	Description
`heartbeat_id`	string	Same ID used in create
`ttl`	int	Time-to-live in milliseconds (e.g. `30000`)

```
client.send_heartbeat("delta_algo_hb_001", ttl_ms=30000)
```

Heartbeat Configuration

```
heartbeat:
  enabled: true
  heartbeat_id: "delta_algo_hb_001"
  ttl_ms: 30000                    # 30 seconds TTL
  send_interval_seconds: 25        # Send every 25 seconds (before TTL expires)
  action_on_miss: "cancel_orders" # Cancel all orders if missed
  unhealthy_count: 1               # Trigger after 1 miss
```

> **IMPORTANT:** The heartbeat runs in a **background daemon thread** and sends every `send_interval_seconds` (25s by default). The TTL is 30s, so there's a 5-second safety margin. If the algo crashes, Delta will auto-cancel all orders after ~30s.

8. Response Format

All Delta Exchange API responses follow this format:

```
{
  "success": true,
  "result": { ... }  // or [...]
}
```

On error:

```
{
  "success": false,
  "error": {
    "code": "error_code_string",
    "context": { ... }
  }
}
```

The algo's client automatically extracts `result` from successful responses.

9. Data Models

9.1 Option Chain Entry (from `/v2/tickers`)

```
{
  "symbol":        "C-BTC-90000-310125",
  "product_id":    12345,
  "contract_type": "call_options",
```

```
  "spot_price":    "95000.50",
  "mark_price":    "1250.75",
  "mark_vol":      "65.5",
  "oi":            150,
  "oi_change_usd_6h": 5000,
  "strike_price":  "90000",
  "greeks": {
    "delta": "0.350",
    "gamma": "0.0001",
    "theta": "-50.25",
    "vega":  "125.30",
    "rho":   "2.15"
  },
  "quotes": {
    "best_bid": "1240.00",
    "best_ask": "1260.00",
    "bid_iv":   "64.8",
    "ask_iv":   "66.2"
  }
}
```

9.2 Product Entry (from `/v2/products`)

```
{
  "symbol":        "C-BTC-90000-310125",
  "product_id":    12345,
  "contract_type": "call_options",
  "tick_size":     "0.5",
  "strike_price":  "90000"
}
```

9.3 Order Object (from `/v2/orders`)

```
{
  "id":              67890,
  "product_id":      12345,
  "size":            1,
  "side":            "sell",
  "order_type":      "limit_order",
  "limit_price":     "1240.00",
  "state":           "open",
  "client_order_id": "opt_1234567890"
}
```

Order states: `open`, `pending`, `closed`, `cancelled`

9.4 Position Object (from `/v2/positions`)

```
{
  "product_id": 12345,
  "size":       -1
}
```

- `size > 0` = long position

- `size < 0` = short position

9.5 Wallet Balance (from `/v2/wallet/balances`)

```
{
  "asset_symbol":      "USDT",
  "available_balance": "10250.50"
}
```

10. Error Handling & Error Codes

Source: [client.py#L12-L19](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/api/client.py#L12-L19)

Exception Hierarchy

```
DeltaAPIError (base)
├── DeltaRateLimitError    → 429 or quota exceeded
├── DeltaAuthError         → Invalid credentials / signature
└── DeltaInsufficientMarginError → Not enough margin
```

Error Codes Handled

Error Code	Exception	Description
`InvalidApiKey`	`DeltaAuthError`	Invalid API key
`SignatureExpired`	`DeltaAuthError`	Timestamp too old
`UnauthorizedApiAccess`	`DeltaAuthError`	Unauthorized access
`Signature Mismatch`	`DeltaAuthError`	Wrong signature
`insufficient_margin`	`DeltaInsufficientMarginError`	Not enough margin
`rate_limit_exceeded`	`DeltaRateLimitError`	429 response
Other	`DeltaAPIError`	General API error

HTTP Status Handling

Status	Behavior
`200`	Parse JSON, extract `result`
`429`	Read `X-RATE-LIMIT-RESET` header, sleep, retry
`5xx`	Retry with backoff (up to max_retries)

11. Configuration Reference

Full configuration from [config.yaml](file:///c:/Users/abhin/OneDrive/Desktop/deltaoptions/delta_algo/config/config.yaml):

Exchange

Parameter	Value	Description
`environment`	`production` / `testnet`	API environment
`base_url_production`	`https://api.india.delta.exchange`	Production URL
`base_url_testnet`	`https://cdn-ind.testnet.deltaex.org`	Testnet URL
`api_key`	—	API key
`api_secret`	—	API secret
`user_agent`	`delta-options-algo-v1`	HTTP User-Agent

Instruments

Parameter	Value	Description
`underlyings`	`[BTC, ETH]`	Assets to trade
`contract_types`	`[call_options, put_options]`	Option types
`expiry_filter`	`weekly`	Prefer weekly expiries
`min_dte`	`2`	Minimum days to expiry
`max_dte`	`8`	Maximum days to expiry

Strategy

Parameter	Value	Description
`name`	`short_strangle`	Strategy type
`target_delta`	`0.15`	Delta of short strikes
`delta_tolerance`	`0.03`	Acceptable delta range
`iv_percentile_min`	`40`	Minimum IV percentile to enter
`use_trend_filter`	`true`	Enable ATR/trend filter
`atr_period`	`14`	ATR calculation period

trend_ma_period	20	Moving average period for trend
max_atr_multiple	2.5	Max ATR ratio for high-vol regime
ic_wing_width_pct	0.05	Iron condor wing width (% of strike)

Entry

Parameter	Value	Description
scan_interval_seconds	60	How often to scan for entries
max_new_trades_per_scan	1	Max new trades per scan cycle
min_premium_usd	10	Minimum net premium to enter
min_bid_ask_spread_pct	0.30	Max bid-ask spread percentage

Exit

Parameter	Value	Description
profit_target_pct	0.35	Close at 35% of max profit
stop_loss_multiple	1.5	Stop if premium reaches 1.5× entry
max_dte_to_force_close	1	Force close ≤ 1 DTE
use_market_on_close	false	Use market orders for exits
close_check_interval_seconds	30	Exit check frequency

Risk

Parameter	Value	Description
max_risk_per_trade_pct	0.20	Max 20% portfolio risk per trade
max_open_positions	4	Max concurrent positions
max_positions_per_underlying	2	Max positions per asset
max_daily_loss_pct	0.05	Halt if -5% daily
max_portfolio_delta	0.20	Max net portfolio delta
max_consecutive_losses	3	Halt after 3 consecutive losses
iv_spike_threshold	1.5	IV spike ratio to block entries
halt_on_circuit_breaker	true	Enable circuit breaker

circuit_breaker_resume_hours	4	Auto-resume after 4 hours

Orders

Parameter	Value	Description
default_order_type	limit	Default order type
limit_order_slippage_ticks	1	Ticks inside best bid
order_timeout_seconds	30	Timeout before market fallback
max_retries	3	API retry attempts
retry_delay_seconds	2	Base retry delay

Rate Limits

Parameter	Value	Description
quota_per_5min	9000	Total quota per 5-minute window
sleep_on_429_seconds	5	Base sleep on rate limit

API Endpoint Summary Table

Method	Endpoint	Auth	Weight	Purpose
GET	/v2/assets	✗	1	List assets (connectivity check)
GET	/v2/products	✗	3	List tradeable products
GET	/v2/products/{symbol}	✗	3	Get single product
GET	/v2/tickers	✗	3	Option chain data with greeks
GET	/v2/tickers/{symbol}	✗	3	Single ticker with greeks
GET	/v2/l2orderbook/{symbol}	✗	3	Level 2 order book
GET	/v2/history/candles	✗	3	OHLC candlestick data
GET	/v2/profile	✓	1	Account profile
GET	/v2/wallet/balances	✓	3	Wallet balances
GET	/v2/positions	✓	3	Positions (filtered)
GET	/v2/positions/margined	✓	3	All margined positions

GET	`/v2/orders`	✓	3	Active orders
GET	`/v2/orders/{id}`	✓	3	Single order by ID
POST	`/v2/orders`	✓	5	Place order
PUT	`/v2/orders`	✓	5	Edit order
DELETE	`/v2/orders`	✓	5	Cancel order
DELETE	`/v2/orders/all`	✓	5	Cancel all orders
POST	`/v2/orders/batch`	✓	25	Batch place orders
POST	`/v2/positions/close_all`	✓	5	Close all positions
POST	`/v2/products/{id}/orders/leverage`	✓	1	Set leverage
POST	`/v2/heartbeat/create`	✓	1	Register deadman switch
POST	`/v2/heartbeat`	✓	1	Send heartbeat